

Universidad Autónoma de Chiriquí

Facultad de Economía

Escuela De Ciencias de Computación



Lic. en Gestión de Tecnología de la Información

¨Programación V – PROG 309¨

Estudiante:

Jairo Chávez

Débora Cisneros

Daniela Pittí

Prof. Alexis Flores

Laboratorio #2

Patrones Maestro-Esclavo y Pipe-Filter

Fecha de Entrega

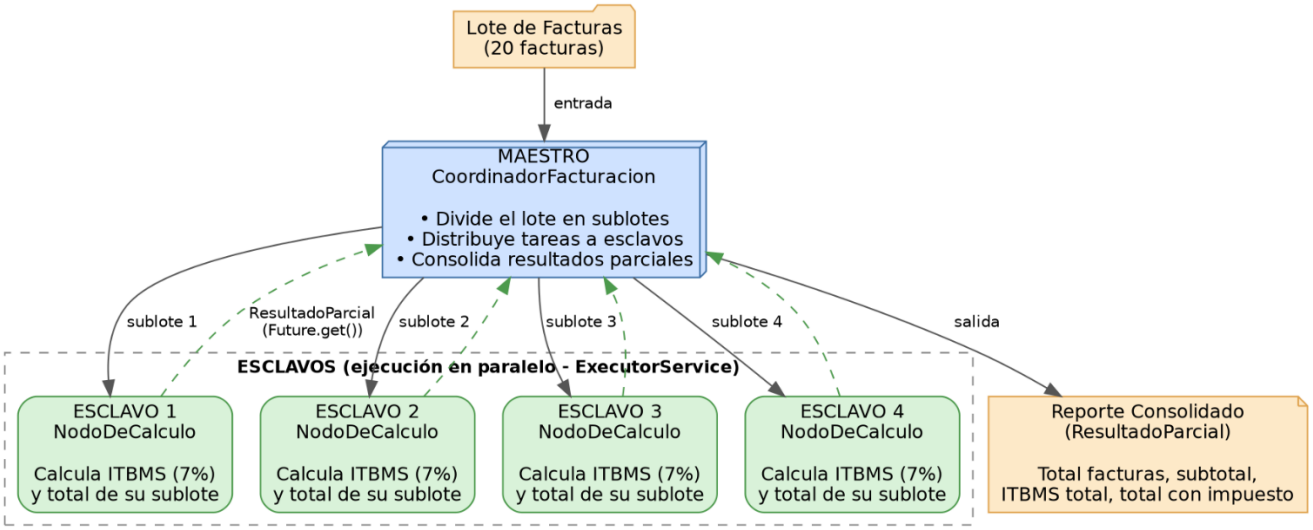
29/08/2026

II Semestre

Justificación Escrita: Patrones Maestro-Esclavo y Pipe-Filter

1. Patrón: Maestro-Esclavo (Master-Slave)

Caso de uso elegido: Procesamiento distribuido de un lote de facturas



1.1 ¿Por qué este patrón encaja con el caso?

El cálculo de cada factura es independiente entre sí, por lo que el maestro puede dividir el lote en sublotes y procesarlos en paralelo con varios esclavos. Esto reduce el tiempo total frente a un cálculo secuencial, ya que no hay dependencias entre subtareas.

1.2 Principal desventaja del patrón en este caso y cómo la mitigarían

La principal desventaja es que el maestro se convierte en un posible cuello de botella y en un punto único de fallo: si el maestro se cae o tarda en repartir el trabajo, ningún esclavo puede avanzar, y si un esclavo falla o tarda demasiado (por ejemplo, un sublote con datos corruptos), el maestro puede quedar bloqueado esperando su Future.get(). Además, si el lote no se divide de forma pareja, algunos esclavos terminan mucho antes que otros y quedan ociosos (desbalance de carga). Para mitigarlo se puede: (1) aplicar tiempos límite (timeouts) a cada Future para no bloquear indefinidamente al maestro, (2) capturar y reintentar sublotes fallidos en otro esclavo, y (3) usar una estrategia de reparto dinámico (por ejemplo, una cola de trabajo compartida) en lugar de una división estática, para que los esclavos más rápidos tomen más sublotes y se balancee mejor la carga.

2. Patrón: Pipe-Filter (Filtro de Tubería)



Caso de uso elegido: Sistema de análisis de logs (lectura → filtrado → agregación → reporte)

2.1 ¿Por qué este patrón encaja con el caso?

El análisis de logs requiere pasos secuenciales bien definidos (filtrar → agrupar → reportar), donde cada etapa solo necesita la salida de la anterior. El patrón permite aislar cada transformación en un filtro independiente, fácil de entender y modificar por separado.

2.2 Principal desventaja del patrón en este caso y cómo la mitigarían

La principal desventaja es el costo de transformar y copiar datos entre cada filtro: cada etapa crea una nueva colección completa (List<LineaLog> filtradas, List<Incidencia> agregadas) en lugar de modificar los datos en el lugar, lo que puede ser ineficiente si el volumen de logs es muy grande, además de que un error en un filtro intermedio obliga a rehacer todo el recorrido de la tubería. Para mitigarlo se puede: (1) procesar los logs en flujo (streaming), por ejemplo con Stream de Java, para evitar mantener en memoria colecciones intermedias completas; (2) agregar manejo de errores por filtro (try/catch y valores centinela o Optional) para que una línea de log corrupta no detenga toda la tubería; y (3) si el volumen crece mucho, dividir la tubería en etapas que se ejecuten en hilos o procesos distintos (pipeline concurrente), de modo que mientras un filtro procesa un lote, el anterior ya está preparando el siguiente.